# FULL STACK DEVELOPMENT
# PROJECT DOCUMENTATION

| | |
|---|---|
| **Date** | 3 October 2026 |
| **Team ID** | SWTID-2026-3849 |
| **Project Name** | Implement Client Script & UI Policy (Incident) |
| Maximum Mark | |

This report follows the supplied documentation format. Its headings are adapted to the actual ServiceNow solution; MERN-specific implementation details are not added because they are not part of the supplied project evidence.

## Team Members and Responsibilities

| Team Member | Assigned Responsibility |
|---|---|
| Neive Joel S J | Creation of oncellEdit client script |
| Nasrin Subaithal N | Creation of on submit client script (save validation) |
| Kanya S | Creation of  UI policy on incident & Testing and configuration |
| Bashera Mariyam A | Creation of UI policy action -Urgency |
| Jeeva Sree MP | Creation of onchange client script |

## 1. Introduction

**Project Title:** Implement Client Script & UI Policy (Incident)

**Purpose of This Document:** This document describes the project objective, proposed solution, platform architecture, setup process, configuration components, testing approach, known limitations, and possible future improvements. It is based on the supplied project planning, proposed solution, solution architecture, and performance testing documents.

**Background:** Incident records are used by service desk teams to record and track issues. If important fields are missing or inconsistent, agents may need to correct records later, and incident priority may not reflect the information entered. Client-side rules can guide users while they work on the Incident form and can prevent selected invalid updates before they are saved.

## 2. Project Overview

**Problem Statement:** The Incident form may accept missing or mismatched information, including combinations of Impact and Urgency that do not align with the intended incident-handling rules. Users may also make quick edits from the Incident list, where mistakes can be repeated. These issues can lead to inconsistent records, additional correction work, and a less predictable user experience.

**Objective:** To configure ServiceNow UI Policies and Client Scripts that guide users, control selected field behaviour, validate data, and reduce avoidable errors when creating or updating incidents.

**Scope:** The project covers a UI Policy, a UI Policy Action for Urgency, and onChange, onSubmit, and onCellEdit Client Scripts on the Incident table. The configured behaviour is checked through manual functional scenarios.

**Key Features:** The UI Policy “High Impact Control” controls the Urgency field. The onChange script sets Urgency to High when Impact is High and displays a message. The onSubmit script validates information before the form is saved. The onCellEdit script checks inline edits made in the Incident list and blocks the specified State update.

**Expected Benefits:** The solution is intended to provide immediate feedback, reduce inconsistent data entry, make field behaviour clearer, and reduce rework for service desk agents. The project documents functional behaviour; it does not yet establish performance under concurrent or peak load.

## 3. Team Members and Responsibilities

**Assignment Note:** The following responsibilities are proposed assignments across the project work. Confirm the allocation with all team members before final submission.

## 4. Solution Architecture

**Presentation Layer:** Users access the ServiceNow Incident form and Incident list through a modern web browser such as Google Chrome, Microsoft Edge, or Firefox. The form collects incident information, while the list provides record viewing and inline editing.

**Platform Layer:** The ServiceNow Now Platform receives browser requests and manages platform-side processing and user sessions. This processing is provided by ServiceNow rather than by a separate project-built Node.js server.

**Authentication and Roles:** Users sign in through the ServiceNow authentication mechanism. Their available actions depend on their assigned roles and permissions. The supplied architecture identifies roles such as admin and itil.

**Client Logic Layer:** A UI Policy and UI Policy Action control the behaviour of fields on the form. Client Scripts use JavaScript and ServiceNow client APIs such as g_form to read or update values, control fields, and display messages. The onCellEdit script handles the documented inline-edit validation.

**Data Layer:** The platform stores incident records in the incident table and configuration records in sys_ui_policy and sys_script_client. These records are managed inside ServiceNow.

**External Services:** No external API integration is specified in the supplied project documents. The described solution relies on built-in ServiceNow platform capabilities.

**Architecture Clarification:** Although the provided documentation template is for MERN full-stack projects, this particular project is a ServiceNow configuration project. It does not require a separate React frontend, Express backend, or MongoDB database.

## 5. Setup Instructions

**Prerequisites:** A ServiceNow Personal Developer Instance (PDI) with Incident Management enabled, a supported web browser, a ServiceNow account, and appropriate administrator/developer permissions to configure UI Policies and Client Scripts.

**Environment Setup:** Sign in to the PDI and confirm that the Incident form and Incident list are available. Use sample incident records for configuration checks so that testing does not affect important operational records.

**Configuration Steps:** 1. Open the UI Policy configuration area and create or review the “High Impact Control” policy. 2. Configure the relevant UI Policy Action for the Urgency field. 3. Create or update the onChange Client Script to respond to Impact changes. 4. Configure the onSubmit Client Script to validate data before saving. 5. Configure the onCellEdit Client Script to check the specified list edit. 6. Save each configuration and test it on the Incident form and list.

**Environment Variables and Installation:** No local .env file, npm installation, or database server setup is specified for this project. Configuration is performed within the ServiceNow instance.

## 6. Folder Structure and Project Components

**Project Structure:** The solution is configured within ServiceNow rather than organised as a local client/server codebase. Therefore, a conventional React client folder and Node.js server folder do not apply.

**Main Configuration Components:** The main artefacts are the UI Policy record, the UI Policy Action for Urgency, and Client Script records for onChange, onSubmit, and onCellEdit behaviour.

**Related Records:** The supplied architecture identifies the incident table for incident data, sys_ui_policy for UI Policy configuration, and sys_script_client for Client Script configuration.

## 7. Running the Application

**How to Access:** Open the ServiceNow PDI URL in a browser and sign in with an account that has the required permissions. Navigate to Incident Management, then open the Incident form or Incident list.

**How the Configuration Runs:** The UI Policy and Client Scripts execute as the user interacts with the Incident form or list. Users can observe field changes, validation messages, and restrictions during the documented workflows.

**Run Commands:** No npm start or other local application command is used. The application and configuration run within the ServiceNow platform.

## 8. API Documentation

**Custom Endpoints:** The supplied project documents do not specify any custom REST API endpoints, request payloads, or response formats. No custom endpoint should be listed as implemented without additional project evidence.

**Client-Side API:** The architecture identifies g_form as a ServiceNow client API used by Client Scripts to work with form fields and messages. The architecture also mentions g_user where user information is needed.

**Integration Notes:** The described solution uses built-in platform functionality and does not require an external API integration.

## 9. Authentication and Authorization

**Authentication:** Users authenticate through the ServiceNow login and platform-managed session. The supplied project documents do not describe a custom authentication module.

**Authorization:** Access to configure UI Policies and Client Scripts requires suitable administrator/developer permissions. Users who work with incident records require the appropriate ServiceNow role, such as itil, according to the instance configuration.

**Tokens and Sessions:** No custom JWT, OAuth implementation, or project-managed session mechanism is documented. Session handling is managed by the ServiceNow platform.

## 10. User Interface

**Incident Form:** The Incident form allows agents to create and update incident records. The configured policy and scripts provide feedback while the user enters or changes field values.

**Incident List:** The Incident list displays incident records and allows inline editing where enabled. The onCellEdit Client Script checks the specified State edit and prevents the documented update.

**Expected User Experience:** When Impact is High, the configured logic sets Urgency to High and makes the field read-only under the specified condition. Validation feedback helps users understand what must be corrected before proceeding.

**Screenshots and Evidence:** Insert screenshots from the ServiceNow instance in this section. The supplied performance-testing document lists evidence for the mandatory Assigned to message, Urgency/Priority behaviour, the change back to Medium impact, and the blocked State list edit.

## 11. Testing

**Testing Method:** The supplied report describes manual functional/behaviour testing in Google Chrome on a cloud-hosted ServiceNow PDI with a single user. No dedicated load-testing tool was used, and load, stress, and spike testing were not performed.

**Scenario 1 — Mandatory Field:** For a High-impact incident with Assigned to left empty (INC0010006), the message “Assigned To is mandatory for High impact incidents.” is shown and Assigned to is treated as mandatory.

**Scenario 2 — High Impact:** When Impact is set to High, Urgency becomes read-only with the value 1 - High, and Priority shows 1 - Critical.

**Scenario 3 — Medium Impact:** When Impact changes from High to Medium, Urgency becomes editable again, Priority changes to 2 - High, and Assigned to is no longer mandatory.

**Scenario 4 — Inline State Edit:** When the State cell is edited directly in the Incident list for INC0008111, an alert explains that State cannot be updated using list editing, and the State value remains unchanged.

**Observed Functional Result:** The supplied performance-testing report states that all four scenarios behaved as designed and that pages responded normally during the manual checks.

**Performance Metrics:** Average response time, maximum response time, throughput, CPU utilisation, memory utilisation, and formal error rate were not measured. These metrics remain pending and should not be reported as passed.

## 12. Sprint Schedule

**Sprint 1 — 1 to 2 October 2026:** Environment setup, UI Policy configuration, UI Policy Action for Urgency, and initial checks of form behaviour.

**Sprint 2 — 2 to 3 October 2026:** onChange behaviour, onSubmit save validation, and clear user-facing validation messages.

**Sprint 3 — 3 October 2026:** onCellEdit validation, functional testing of the configuration, and completion of project documentation.

## 13. Known Issues and Limitations

**Unmeasured Performance:** The current evidence comes from single-user manual checks. Response time, throughput, CPU utilisation, memory utilisation, and formal error rate have not been measured.

**Concurrent Usage:** The system's behaviour under many simultaneous users is unknown because load, stress, and spike testing were outside the reported test scope.

**Platform Dependency:** The configuration depends on access to the ServiceNow platform, a working Incident Management setup, and appropriate permissions.

**Evidence Dependency:** Screenshots should be added to the final report if required by the evaluator. Performance metrics should be updated only after measurements have been collected.

## 14. Future Enhancements

**Expanded Testing:** Run timed checks and controlled load/stress tests if performance evaluation is required. Record the environment, number of users, duration, and actual measurements so results can be reproduced.

**Broader Validation:** Test additional combinations of Impact and Urgency, empty required fields, and invalid list edits to confirm the rules behave consistently across relevant scenarios.

**Maintenance:** Keep the Client Scripts focused on clearly defined behaviours, document each policy condition, and maintain a repeatable set of test cases and screenshots.

**Documentation:** Update the report with final screenshots, measured performance results, and confirmed team responsibilities before submission.